

REINFORCEMENT LEARNING LAB

WEEK – 1

Design and implement a program that simulates an n-armed bandit problem, where different actions have unknown reward probabilities.

CODE:

```
import random
import numpy as np
import matplotlib.pyplot as plt
n_arms = 10
true_probabilities = np.random.rand(n_arms)
Q = np.zeros(n_arms)
N = np.zeros(n_arms)
epsilon = 0.1
steps = 1000
rewards = []
print("True reward probabilities: ")
print(true_probabilities)
for step in range(steps):
    if random.random() < epsilon:
        action = random.randint(0, n_arms-1)
    else:
        action = np.argmax(Q)
    reward = 1 if random.random() < true_probabilities[action] else 0
    N[action] += 1
    Q[action] = Q[action] + (1/N[action]) * (reward - Q[action])
    rewards.append(reward)
print("\n Estimated Action Values: ")
print(Q)
print("\n Best Action Found: ", np.argmax(Q))
avg_rewards = np.cumsum(rewards) / (np.arange(steps) + 1)
plt.plot(avg_rewards)
plt.xlabel("Steps")
plt.ylabel("Average Reward")
plt.title("n_Armed Bandit Performance")
plt.grid(True)
plt.show()
```

REINFORCEMENT LEARNING LAB

OUTPUT:

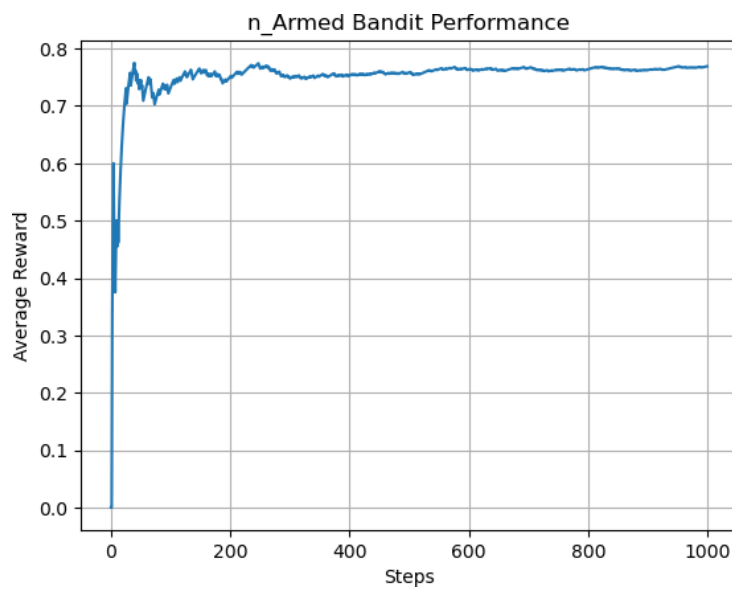
True reward probabilities:

[0.00897581 0.77685707 0.19067037 0.4339314 0.61097859 0.31225136
0.82313119 0.2740083 0.61191243 0.45014776]

Estimated Action Values:

[0. 0.80446927 0.11111111 0.33333333 0.75 0.2
0.80888291 0.16666667 0.33333333 0.27272727]

Best Action Found: 6



REINFORCEMENT LEARNING LAB

WEEK – 2

Develop a reinforcement learning program that implements Samuel's Checkers Player

CODE:

```
import random
weights = {
    "my_pieces":1.0,
    "opponent_pieces":-1.0,
    "my_kings":2.0,
    "opponent_kings":-2.0
}
learning_rate = 0.1
def evaluate(state):
    return(
        weights["my_pieces"]*state["my_pieces"]+["opponent_pieces"]*state["my_pieces"]+
        weights["my_kings"]*state["my_kings"]+["opponent_kings"]*state["my_kings"]
    )
def update_weights(state, target):
    prediction = evaluate(state)
    error = target-prediction
    for features in state:
        weights[feature] += learning_rate*error*state[feature]
    training_data = [
        ("my_pieces":12, "opponent_pieces":10, "my_kings":2, "opponent_kings":1},1),
        ("my_pieces":8, "opponent_pieces":11, "my_kings":1, "opponent_kings":2},-1),
        ("my_pieces":10, "opponent_pieces":10, "my_kings":2, "opponent_kings":2},0),
        ("my_pieces":11, "opponent_pieces":9, "my_kings":3, "opponent_kings":1},1),
    ]
    print("Initial weights: ")
    print(weights)
    test_state = {
        "my_pieces":11,
        "opponent_pieces":9,
        "my_kings":2,
        "opponent_kings":1
    }
    score = evaluate(test_state)
    print("\n Test Board: ", test_state)
    print("Evaluation Score: ",score)
    if score>0:
        print("Predicted Result: Good position")
```

G. NARAYANAMMA INSTITUTE OF TECHNOLOGY AND SCIENCE

REINFORCEMENT LEARNING LAB

```
elif score<0:  
    print("Predicted Result: Bad position")  
else:  
    print("Predicted Result: Neutral position")
```

OUTPUT:

Initial weights:

{'my_pieces': 1.0, 'opponent_pieces': -1.0, 'my_kings': 2.0, 'opponent_kings': -2.0}

Updated weights:

{'my_pieces': 25955.382000000012, 'opponent_pieces': 21037.758000000001, 'my_kings': 7163.5060000000003, 'opponent_kings': 2230.1820000000001}

Test Board: {'my_pieces': 11, 'opponent_pieces': 9, 'my_kings': 2, 'opponent_kings': 1}

Evaluation Score: 491406.2180000002

Predicted Result: Good position

REINFORCEMENT LEARNING LAB

WEEK – 3

Write a program to implement value iteration and policy iteration algorithms for solving Markov Decision Problems. Use a simple grid world environment and calculate the optimal value function and policy.

CODE:

```
import numpy as np

ROWS, COLS = 3, 3

gamma = 0.9

theta = 1e-4

ACTIONS = [(-1,0), (1,0), (0,-1), (0,1)]

action_names = ['U','D','L','R']

terminal = (2,2)

def next_state(state, action):

    if state == terminal:

        return state

    r,c = state

    dr,dc = action

    nr = min(max(r+dr,0), ROWS-1)

    nc = min(max(c+dc,0), COLS-1)

    return (nr,nc)

def reward(state):

    if state == terminal:

        return 0

    return -1

def value_iteration():

    V = np.zeros((ROWS,COLS))

    while True:

        delta = 0

        for r in range(ROWS):
```

G. NARAYANAMMA INSTITUTE OF TECHNOLOGY AND SCIENCE

REINFORCEMENT LEARNING LAB

```
for c in range(COLS):
    if (r,c) == terminal:
        continue
    old_value = V[r,c]
    values = []
    for action in ACTIONS:
        ns = next_state((r,c),action)
        values.append(reward((r,c)) + gamma*V[ns[0],ns[1]])
    V[r,c] = max(values)
    delta = max(delta,abs(old_value-V[r,c]))
    if delta < theta:
        break
policy = np.full((ROWS,COLS),"",dtype=object)
for r in range(ROWS):
    for c in range(COLS):
        if (r,c) == terminal:
            policy[r,c] = 'T'
            continue
        best_action = None
        best_value = -float('inf')
        for i,action in enumerate(ACTIONS):
            ns = next_state((r,c),action)
            value = reward((r,c)) + gamma*V[ns[0],ns[1]]
            if value > best_value:
                best_value = value
                best_action = action_names[i]
        policy[r,c] = best_action
return V,policy
def policy_iteration():
```

G. NARAYANAMMA INSTITUTE OF TECHNOLOGY AND SCIENCE

REINFORCEMENT LEARNING LAB

```
policy = np.random.randint(0,4,size=(ROWS,COLS))
V = np.zeros((ROWS,COLS))
while True:
    while True:
        delta = 0
        for r in range(ROWS):
            for c in range(COLS):
                if (r,c) == terminal:
                    continue
                old = V[r,c]
                action = ACTIONS[policy[r,c]]
                ns = next_state((r,c),action)
                V[r,c] = reward((r,c)) + gamma*V[ns[0],ns[1]]
                delta = max(delta,abs(old-V[r,c]))
            if delta < theta:
                break
        stable = True
        for r in range(ROWS):
            for c in range(COLS):
                if (r,c) == terminal:
                    continue
                old_action = policy[r,c]
                action_values = []
                for action in ACTIONS:
                    ns = next_state((r,c),action)
                    action_values.append(reward((r,c)) + gamma*V[ns[0],ns[1]])
                best_action = np.argmax(action_values)
                policy[r,c] = best_action
            if old_action != best_action:
```

G. NARAYANAMMA INSTITUTE OF TECHNOLOGY AND SCIENCE

REINFORCEMENT LEARNING LAB

```
stable = False

if stable:

    break

display_policy = np.full((ROWS, COLS), "", dtype=object)

for r in range(ROWS):

    for c in range(COLS):

        if (r,c) == terminal:

            display_policy[r,c] = 'T'

        else:

            display_policy[r,c] = action_names[policy[r,c]]

return V, display_policy

V_vi, P_vi = value_iteration()

print("VALUE ITERATION")

print("\nOptimal Value Function:")

print(np.round(V_vi, 2))

print("\nOptimal Policy:")

print(P_vi)

V_pi, P_pi = policy_iteration()

print("\nPOLICY ITERATION")

print("\nOptimal Value Function:")

print(np.round(V_pi, 2))

print("\nOptimal Policy:")

print(P_pi)
```

G. NARAYANAMMA INSTITUTE OF TECHNOLOGY AND SCIENCE

REINFORCEMENT LEARNING LAB

OUTPUT:

VALUE ITERATION

Optimal Value Function:

[[-3.44 -2.71 -1.90]

[-2.71 -1.90 -1.00]

[-1.90 -1.00 0.00]]

Optimal Policy:

['D' 'D' 'D']

['R' 'D' 'D']

['R' 'R' 'T']]

POLICY ITERATION

Optimal Value Function:

[[-3.44 -2.71 -1.90]

[-2.71 -1.90 -1.00]

[-1.90 -1.00 0.00]]

Optimal Policy:

['D' 'D' 'D']

['R' 'D' 'D']

['R' 'R' 'T']]

REINFORCEMENT LEARNING LAB

WEEK – 4

Write a program to implement the Monte Carlo prediction algorithm to estimate the value function for a given policy. Apply it to a simple environment, such as the Blackjack game, to evaluate the performance of the policy.

CODE:

```
import random

from collections import defaultdict

def draw_card():
    card = random.randint(1,13)
    return min(card,10)

def draw_hand():
    return [draw_card(),draw_card()]

def usable_ace(hand):
    return 1 in hand and sum(hand)+10 <= 21

def hand_value(hand):
    value = sum(hand)
    if usable_ace(hand):
        value += 10
    return value

def is_bust(hand):
    return hand_value(hand) > 21

def policy(player_sum):
    if player_sum > 20:
        return "stick"
    else:
        return "hit"

def generate_episode():
    player = draw_hand()
```

G. NARAYANAMMA INSTITUTE OF TECHNOLOGY AND SCIENCE

REINFORCEMENT LEARNING LAB

```
dealer = draw_hand()

states = []

while True:

    state = (hand_value(player),dealer[0])

    states.append(state)

    action = policy(hand_value(player))

    if action == "stick":

        break

    player.append(draw_card())

    if is_bust(player):

        return states,-1

while hand_value(dealer) < 17:

    dealer.append(draw_card())

player_value = hand_value(player)

dealer_value = hand_value(dealer)

if is_bust(dealer):

    reward = 1

elif player_value > dealer_value:

    reward = 1

elif player_value < dealer_value:

    reward = -1

else:

    reward = 0

return states,reward

def monte_carlo_prediction(epochs=10000):

    returns = defaultdict(list)

    V = defaultdict(float)

    for _ in range(epochs):

        states,reward = generate_episode()
```

G. NARAYANAMMA INSTITUTE OF TECHNOLOGY AND SCIENCE

REINFORCEMENT LEARNING LAB

```
visited = set()

for state in states:

    if state not in visited:

        returns[state].append(reward)

        V[state] = sum(returns[state])/len(returns[state])

        visited.add(state)

return V

V = monte_carlo_prediction()

print("Monte Carlo Prediction - Blackjack")

print("\nEstimated Value Function:")

for state,value in sorted(V.items()):

    print(f'State {state}: {value:.3f}')
```

OUTPUT:

Estimated Value Function:

```
State (4,1): -0.667
State (4,3): 0.000
State (4,4): -1.000
State (4,5): -1.000
State (4,6): -1.000
State (4,7): 0.000
State (4,9): -1.000
State (4,10): -0.833
State (5,1): -1.000
State (5,2): -0.200
State (5,3): 0.000
State (5,4): -1.000
State (5,5): -1.000
State (5,6): -0.333
```

REINFORCEMENT LEARNING LAB

State (5,7): -1.000

State (5,8): -1.000

State (5,9): -0.333

State (5,10): -0.760

State (6,1): -0.714

State (6,2): -0.500

State (6,3): -0.333

State (6,4): -0.667

State (6,5): -0.500

State (6,6): -0.250

REINFORCEMENT LEARNING LAB

WEEK – 5

Write a program to implement the Q-Learning algorithm to find an optimal policy for navigating the grid. Users can experiment with different learning rates and exploration strategies.

CODE:

```
import numpy as np
import random

alpha = 0.1
gamma = 0.9
epsilon = 0.2
episodes = 1000
max_steps = 100
ROWS = 4
COLS = 4
start = (0,0)
goal = (3,3)
actions = ["Up","Down","Left","Right"]
moves = [(-1,0),(1,0),(0,-1),(0,1)]
Q = np.zeros((ROWS,COLS,len(actions)))

def next_state(state,action):
    r,c = state
    dr,dc = moves[action]
    nr = max(0,min(ROWS-1,r+dr))
    nc = max(0,min(COLS-1,c+dc))
    return (nr,nc)

def reward(state):
    if state == goal:
        return 100
```

REINFORCEMENT LEARNING LAB

```
    return -1

for episode in range(epochs):

    state = start

    for step in range(max_steps):

        if random.random() < epsilon:

            action = random.randint(0,3)

        else:

            action = np.argmax(Q[state[0],state[1]])

        new_state = next_state(state,action)

        r = reward(new_state)

        old_q = Q[state[0],state[1],action]

        max_future_q = np.max(Q[new_state[0],new_state[1]])

        Q[state[0],state[1],action] = old_q + alpha*(r + gamma*max_future_q - old_q)

        state = new_state

        if state == goal:

            break

    if (episode+1) % 100 == 0:

        print(f'Episode {episode+1} completed')

print("\nFinal Q-Table:\n")

for i in range(ROWS):

    for j in range(COLS):

        print(f'State ({i},{j}): {Q[i][j]}')

symbols = {0:"↑",1:"↓",2:"←",3:"→"}

print("\nOptimal Policy:\n")

for i in range(ROWS):

    row = []

    for j in range(COLS):

        if (i,j) == goal:

            row.append("G")
```

G. NARAYANAMMA INSTITUTE OF TECHNOLOGY AND SCIENCE

REINFORCEMENT LEARNING LAB

else:

```
best_action = np.argmax(Q[i][j])
```

```
row.append(symbols[best_action])
```

```
print(" ".join(row))
```

OUTPUT:

Episode 100 completed

Episode 200 completed

Episode 300 completed

Episode 400 completed

Episode 500 completed

Episode 600 completed

Episode 700 completed

Episode 800 completed

Episode 900 completed

Episode 1000 completed

Final Q-Table:

State (0,0): [48.32457955 53.49423797 48.36926292 54.9539]

State (0,1): [54.73668796 62.171 48.02657193 61.39429249]

State (0,2): [34.19084486 70.17236505 25.45524488 34.34829941]

State (0,3): [-0.29701 79.02242884 16.55159585 11.68922099]

State (1,0): [31.25272318 7.78374233 9.72622712 62.15976888]

State (1,1): [54.72447895 53.63203382 54.39969432 70.19]

State (1,2): [59.69637451 78.90734935 62.11365559 79.1]

State (1,3): [69.52704399 89. 70.12608769 78.89225854]

State (2,0): [9.70529633 -0.50444695 -0.52021951 34.57288833]

G. NARAYANAMMA INSTITUTE OF TECHNOLOGY AND SCIENCE

REINFORCEMENT LEARNING LAB

State (2,1): [61.95665479 15.65257092 7.9717297 21.13772697]

State (2,2): [42.87419815 45.34483981 13.6366465 88.99911127]

State (2,3): [79.04162064 100. 78.76034506 88.82192466]

State (3,0): [-0.2881 -0.29701 -0.29701 2.21203976]

State (3,1): [5.2588037 1.07076183 -0.24149818 46.74727842]

State (3,2): [7.81790351 19.63316946 3.23310734 91.13706188]

State (3,3): [0. 0. 0. 0.]

Optimal Policy:

→ ↓ ↓ ↓

→ → → ↓

→ ↑ → ↓

→ → → G